

Working with SQL in SQLite

Structured Query Language (SQL)

Create records. Ask for them. Change what persists.

Gregory S. DeLozier, PhD

A Fresh Practice Database

```
mkdir sqlite_practice  
cd sqlite_practice  
sqlite3 pets.db
```

Which directory contains this database?

Which Program Reads the Command?

Input	Who reads it?
<code>sqlite3 pets.db</code>	Operating-system shell
<code>.tables</code>	SQLite command-line shell
<code>SELECT * FROM pet;</code>	SQLite database engine

The `$` and `sqlite>` prompts are not input.

Inspecting the Database

Transcript excerpt:

```
sqlite> .schema  
sqlite> .tables  
sqlite> .headers on  
sqlite> .mode column
```

Definitions, table names, and readable output

Creating the Pet Table

```
CREATE TABLE pet (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    name TEXT NOT NULL,  
    kind TEXT NOT NULL,  
    age INTEGER,  
    food TEXT  
);
```

The table exists. There are no pets yet.

Inserting Dorothy

```
INSERT INTO pet (name, kind, age, food)
VALUES ('Dorothy', 'dog', 11, 'peanut butter');
```

name	kind	age	food
Dorothy	dog	11	peanut butter

Where does id come from?

Reading the Pets

After inserting Sandy and Whiskers:

```
SELECT * FROM pet;
```

id	name	kind	age	food
1	Dorothy	dog	11	peanut butter
2	Sandy	cat	11	tuna
3	Whiskers	hamster	11	hamster chow

Choosing Columns and Rows

```
SELECT name, food  
FROM pet  
WHERE kind = 'cat';
```

name	food
Sandy	tuna

Which part chooses columns? Which part chooses rows?

Deleting the Cats

```
SELECT * FROM pet WHERE kind = 'cat';  
DELETE FROM pet WHERE kind = 'cat';
```

Remaining identifiers: **1 and 3**

What would happen without WHERE?

A Query from the Shell

Leave SQLite with `.quit`, then run:

```
sqlite3 pets.db -header -column \  
"SELECT * FROM pet WHERE name = 'Whiskers';"
```

id	name	kind	age	food
3	Whiskers	hamster	11	hamster chow

Several Lines of Shell Input

```
sqlite3 pets.db <<'SQL'  
SELECT *  
FROM pet;  
SQL
```

The closing SQL marks the end of the input.

Restoring Sandy for the Sorting Examples

```
INSERT INTO pet (id, name, kind, age, food)  
VALUES (2, 'Sandy', 'cat', 11, 'tuna');
```

The transcript skips this step.

Sorting the Result

```
SELECT name, kind FROM pet ORDER BY kind;
```

name	kind
Sandy	cat
Dorothy	dog
Whiskers	hamster

What order would `ORDER BY food produce?`